k – Nearest Neighbours (k-NN) Algorithm

k – Nearest Neighbours (k-NN) Algorithm

- A **supervised** learning algorithm.
- A **non-parametric** learning algorithm: cannot be characterized by a bounded set of parameters.
- An **instance-based learning algorithm** (also called as **lazy learning** algorithm): Stores the entire training data and uses it directly to make predictions.
- Commonly used for classification and regression.

Applications:

- Text categorization
- Image recognition
- Recommender systems
- Medical diagnosis

Key idea: A test point's label is determined by the majority label of its k nearest neighbours.

Steps:

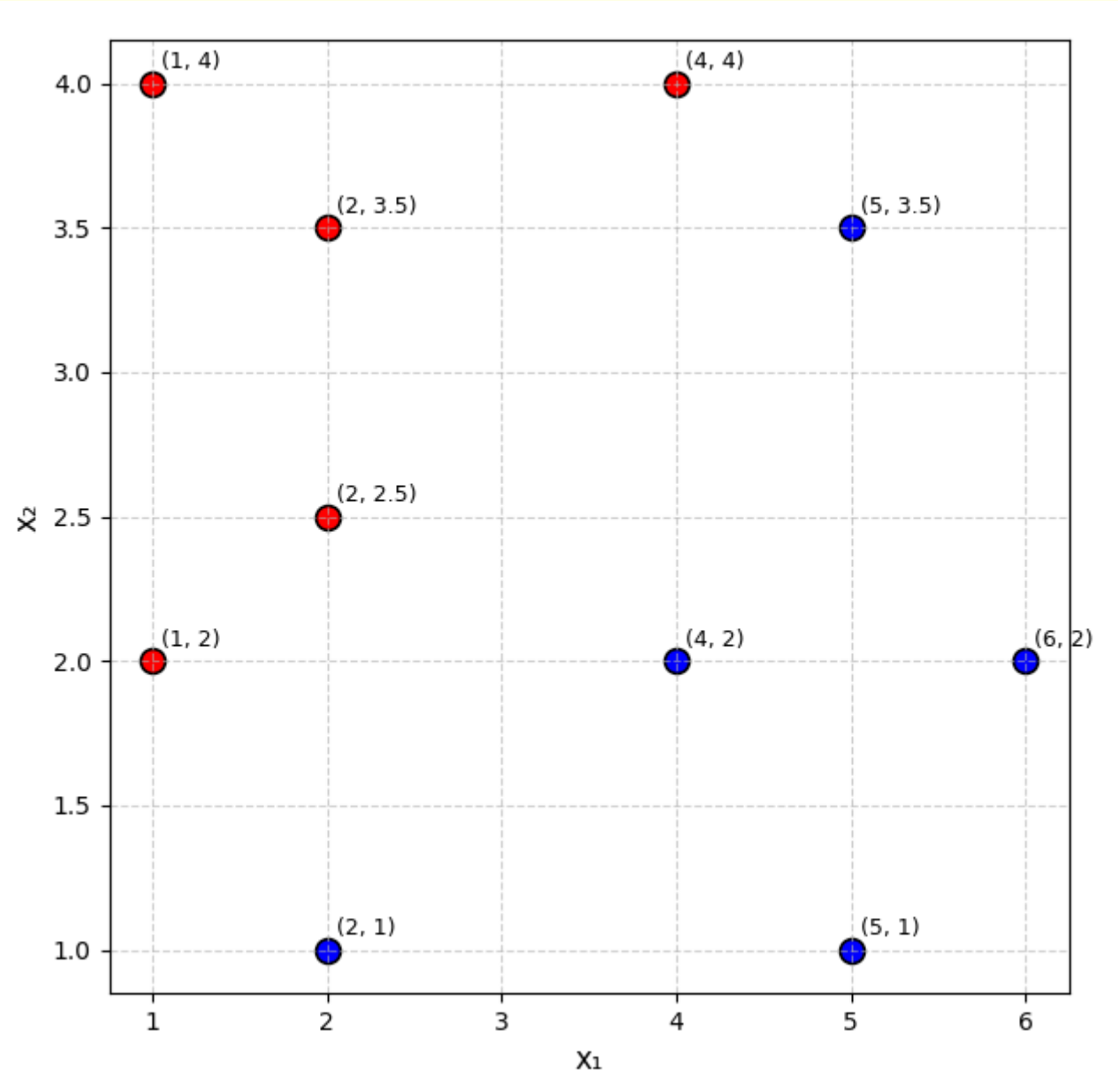
- Store training data $(x_i,\ y_i)$.
- For a new test point x_{test}:
  - Compute distance to all training points.
  - Identify the k nearest neighbours.
- For classification: Assign majority class among the k nearest neighbours.

  For regression: Assign (weighted) average of the k nearest neighbours.

Example:

Suppose we have the data points in two dimensions with class labels as below:

Point	x_1	x_2	Class
A	1	2	Red
B	2	3.5	Red
C	2	2.5	Red
D	4	4	Red
E	1	4	Red
F	5	3.5	Blue
G	5	1	Blue
H	4	2	Blue
I	6	2	Blue
J	2	1	Blue

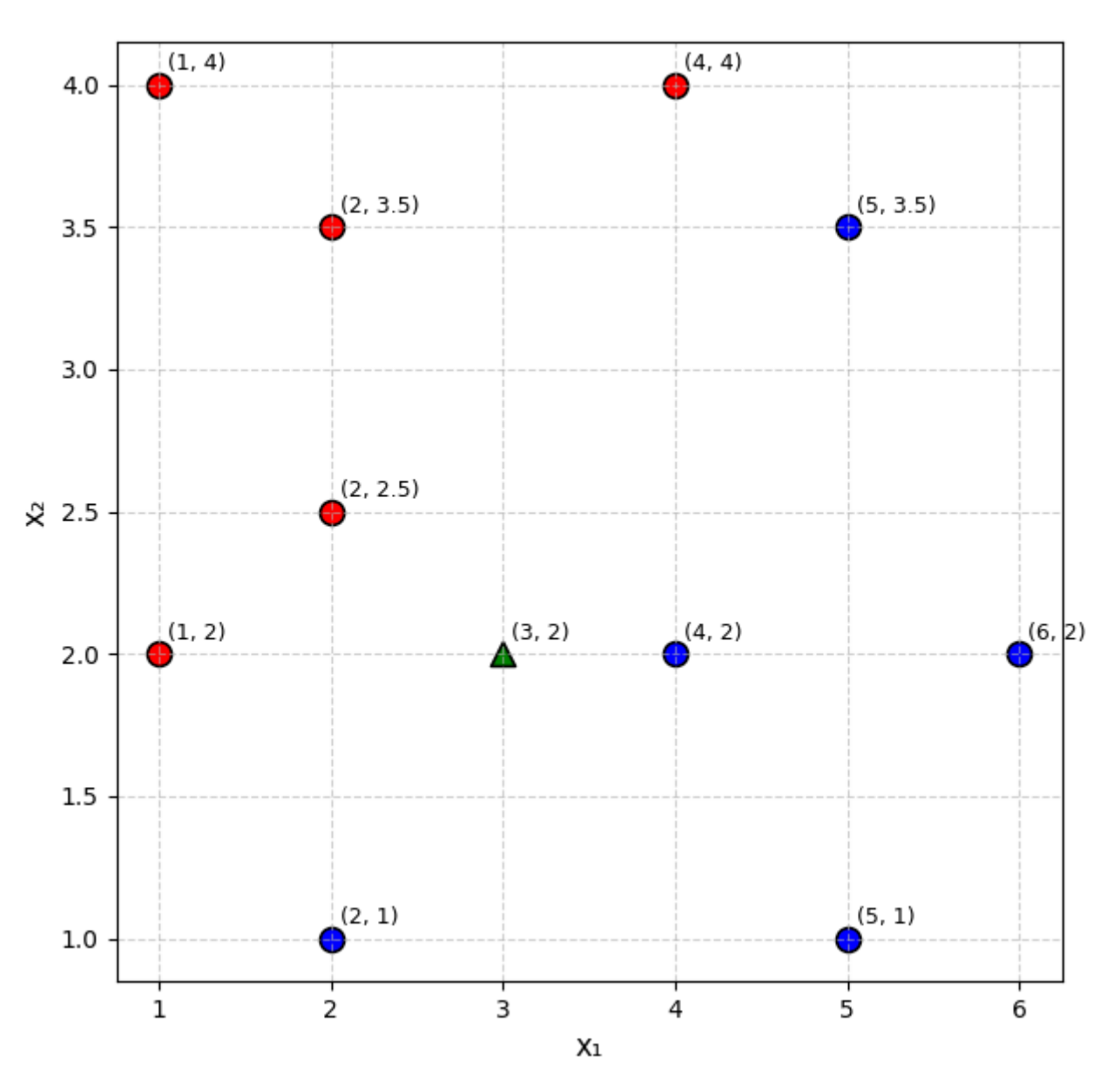

(1, 4)
(4, 4)
(2, 3.5)
(5, 3.5)
(2, 2.5)
(1, 2)
(4, 2)
(6, 2)
(2, 1)
(5, 1)
4.0
3.5
3.0
2.5
2.0
1.5
1.0
1
2
3
4
5
6
X2
X1

Classify the new point P(3, 2) using k-NN algorithm

(i) with k = 1

(ii) with k = 3

(iii) with k = 5.


(1, 4)
(4, 4)
(2, 3.5)
(5, 3.5)
(2, 2.5)
(1, 2)
(3, 2)
(4, 2)
(6, 2)
(2, 1)
(5, 1)
4.0
3.5
3.0
2.5
2.0
1.5
1.0
X2
1
2
3
4
5
6
X1

Finding Euclidean distance from P(3, 2) to all the training points:

Point	Coordinates	Distance from P(3, 2)
A	(1, 2)	2.000
B	(2, 3.5)	1.803
C	(2, 2.5)	1.118
D	(4, 4)	2.236
E	(1, 4)	2.828
F	(5, 3.5)	2.500
G	(5, 1)	2.236
H	(4, 2)	1.000
I	(6, 2)	3.000
J	(2, 1)	1.414

k = 1:
The nearest neighbour to the point P(3, 2) is H(4, 2), whose class is Blue.
Predicted class for P(3, 2) is **Blue**.

k = 3:
The three nearest neighbours to the point P(3, 2) are H(4, 2), C(2, 2.5) and J(2, 1).
Their classes are Blue, Red, Blue respectively.
Predicted class for P(3, 2) is **Blue** [the majority class].

k = 5:
The five nearest neighbours to the point P(3, 2) are H(4, 2), C(2, 2.5), J(2, 1), B(2, 3.5) and A(1, 2).
Their classes are Blue, Red, Blue, Red, Red respectively.
Predicted class for P(3, 2) is **Red** [the majority class].

Distance Metrics

- Minkowski distance (L^p norm):

$$L^p(x_i, x_j) = \left(\sum_k |x_{i,k} - x_{j,k}|^p \right)^{\frac{1}{p}}$$

Commonly used:

$p = 1$: Manhattan distance

$p = 2$: Euclidean distance

Euclidean distance is often used when the features are similar (such as length, width, etc.).

Manhattan distance is often used when the features are dissimilar (such as age, weight, etc.).

Other possible distance metrics:

- **Cosine similarity/distance:**

  Used in text data/ high-dimensional sparse data – when the directions of vectors are more important than their magnitudes.

  Cosine similarity between two vectors $\vec{v}$ and $\vec{w}$ is given by

$$\cos\theta = \frac{\vec{v} \cdot \vec{w}}{\|\vec{v}\|\|\vec{w}\|}$$

  and cosine distance is computed as

$$d_{cos}(\vec{v}, \vec{w}) = 1 - \cos\theta.$$

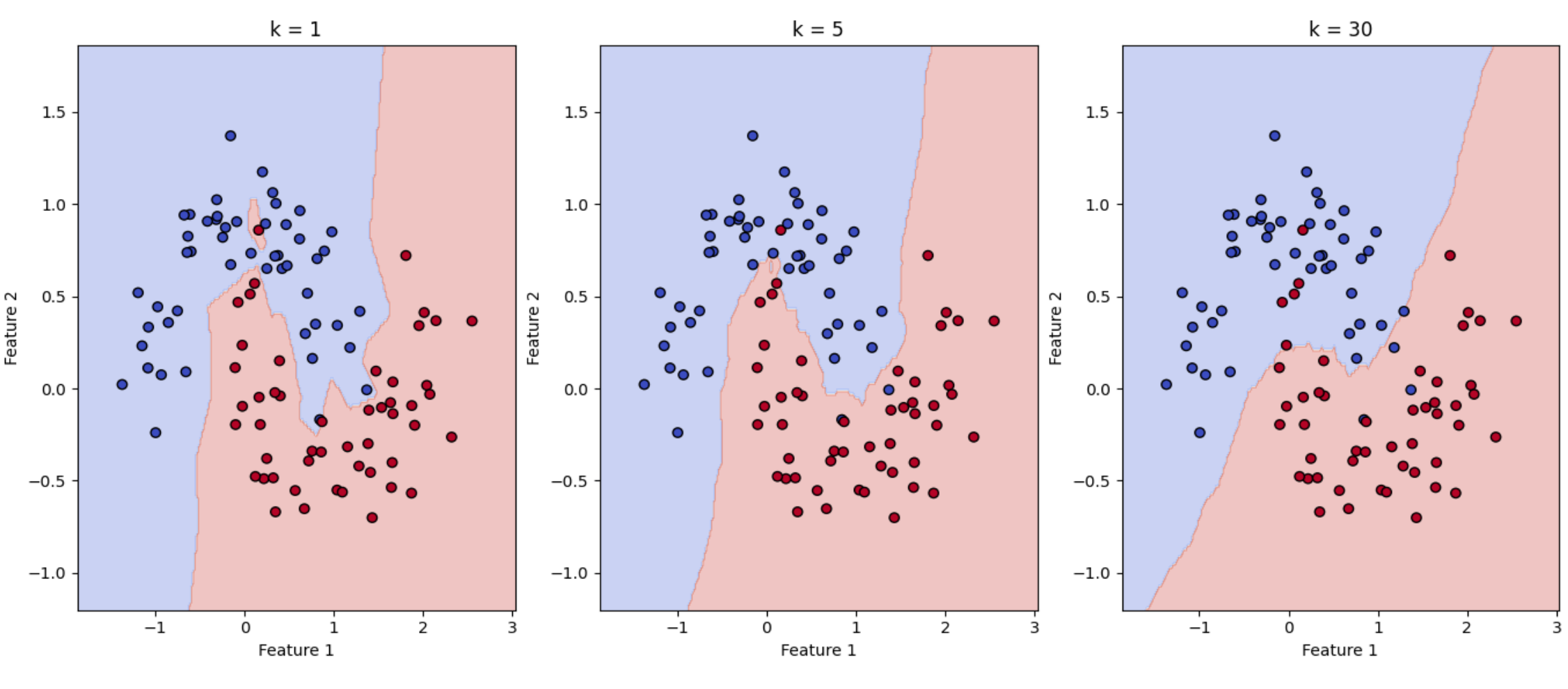
- **Hamming distance:**
  - Used to measure how different two vectors are, when the features are binary or categorical.
  - Computes the number of positions at which the two vectors differ (that is, the number of features on which the two points differ).

For two vectors $x = (x_1, x_2, \dots, x_n)$ and $y = (y_1, y_2, \dots, y_n)$,

$$d_H(x, y) = \sum_{i=1}^{n} \mathbf{1}\{x_i \neq y_i\},$$

where $\mathbf{1}\{\cdot\}$ is the indicator function.

How does the choice of k affect the model?



- Small value of k → Low Bias, High Variance → Overfitting.
- Large value of k → High Bias, Low Variance → Underfitting.
- **Cross-validation** is used to choose the best value of k for a given data.

Need for Normalization of Data in k-NN

- k-NN relies on distance metrics to find nearest neighbours.

- If features have different scales, the feature with the largest numerical range dominates the distance calculation.
  For example,
  Height in cm ranges has a range 150-200.
  Annual income in the US in $ has a range 30000-200000.
  If data is used as it is, then distance will be dominated by the income feature.

- Without normalization, k-NN may give misleading neighbours.

Common types of normalization:

1. Min-Max Scaling:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

This rescales the data to [0,1].

2. Z-Score Normalization:

$$x' = \frac{x - \mu}{\sigma}$$

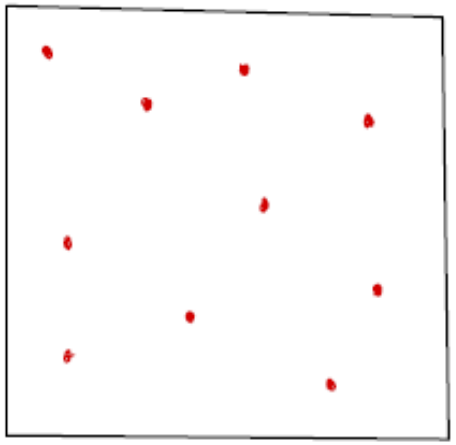
This converts the data to have zero mean, unit variance.

The problem with using k-NN in high dimensions

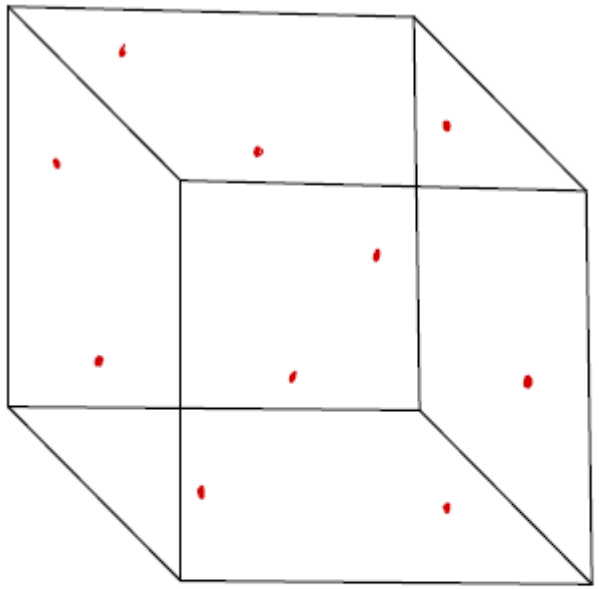
In low dimensional spaces with a large amount of data, k-NN algorithm works well.

As the number of dimensions (features) increases, the nearest neighbours are usually not very near!

Intuition:



A 2D - hypercube (i.e., square) with 10 uniformly distributed data points.



A 3D - hypercube (i.e., cube) with 10 uniformly distributed data points.

- The increase in the dimensions of the feature space increases the sparsity of the data points. This makes the k “nearest” neighbours less meaningful in higher dimensions, thereby leading to incorrect predictions for the query points.

- The increased number of incorrect predictions leads to poor performance in higher dimensions.

- Also, computing distances in higher dimensions involves more calculations. Hence k-NN is computationally intensive in higher dimensions.

This is called the **“Curse of Dimensionality”**.

This needs to be overcome by using dimension reduction techniques such as Principal Component Analysis (PCA).

Advantages of k-NN Algorithm:

- Simple to train
- No training time
- Adapts to new data easily

Disadvantages of k-NN Algorithm:

- Slow prediction
- Memory intensive
- Sensitive to irrelevant features and scaling
- Performs poorly in high-dimensional spaces